SCREAMINGFACE · URL4 · DESIGN DECISION

url4 topology — node, host, discovery, addressing, transport

Sharp definitions before the Engine grows further

STATUS	proposed — owner review; one decision left open in §5; aligned with the url4-refactor drafts 2026-09-08 (§12)
WORK ITEM	OME-1110
DATE	2026-09-04
OWNER	Sergey Bershadsky (execution and telemetry architecture)
GRAMMAR OWNER	Kevin McDonough (URL4 spec, Parts A/B)
SPEC POINTERS	URL4.ai Specification v0.5 DRAFT — Parts A/B (main) and Parts C–I drafts (url4-refactor branch)

Read §0 first. Everything else is the reasoning behind one line of that table.
One decision is left open for the owner, in §5.

0. The eight answers

This document fixes the words we use for the pieces of url4. It is short on purpose. Each section states a definition, says where the spec agrees or is silent, and stops.

#	QUESTION	ANSWER
1	What is a node?	A stateless function behind a path. It takes `(context)!intent` and returns a result. Every node can evaluate a full url4 expression.
2	What is a host?	The origin (`scheme://authority`) that mounts one or more nodes at paths and answers discovery. `/` is its default node.
3	Swarm? Composition?	Not protocol words. The expression **is** the composition. The hosts it touches are its request tree. “Swarm” is only a deployment word for one operator’s hosts.
4	`.well-known` or OPTIONS?	Three mechanisms now: the host document (Part G §27.1), the `Capabilities` response header (also §27.1), and OPTIONS (ours). §5 documents all three; the owner picks there.
5	`/name` vs `url4://name` ?	`/name` is a node mounted on the host that is evaluating. `url4://name` is host `name` ’s default node `/` . Spec Part B §5.4 already says this.
6	Where does an ensemble run?	Wherever an evaluator runs: the SDK on a laptop, or a host. A local host may mount remote nodes under local names (proxy mounts).
7	Can I test a host first?	One fetch of the host’s capabilities document, or one OPTIONS per node (§5). A dry-run **plan** on the host is reopened as a question (§10).
8	Streaming fallback?	One GET asks for the richest mode; the node answers with the best it has: WebSocket, then SSE, then sync (§6). Sync is the only MUST. Async is the spec’s third `delivery` value.
9	Text in, text out?	A habit, not a rule. Every edge carries a **typed payload** (text, image, audio, video, embeddings) named by its media type. Text is the default. No grammar change (§8).

What changes against today

- The Engine becomes a url4 **host**. Today no url4 node is reachable over HTTP; the SDK’s node-to-node code is unused.
- Every node speaks the same GET. The node picks the richest delivery it supports: **WebSocket → SSE → sync**, in one round trip. Sync is the only MUST.
- Two spec words move: the spec’s Node becomes our Host; the spec’s Endpoint becomes our Node. Appendix A asks Kevin for the rename.
- A node is a **universal inference processor**: image generation, speech, video and multimodal ensembles become ordinary expressions on the same engine, telemetry and cost accounting (§8).

1. Terms

Eight words. Each has one meaning.

OUR WORD	MEANING	SPEC WORD TODAY	IN THE SDK	IN THE ENGINE
Node	A stateless function at a path. Accepts `GET <path>?q=<expr>` , evaluates it, returns a result plus envelope.	Endpoint (Part A §1.4)	an entry in `Url4Node` ’s endpoint registry	a route such as `/anthropic/<model>`

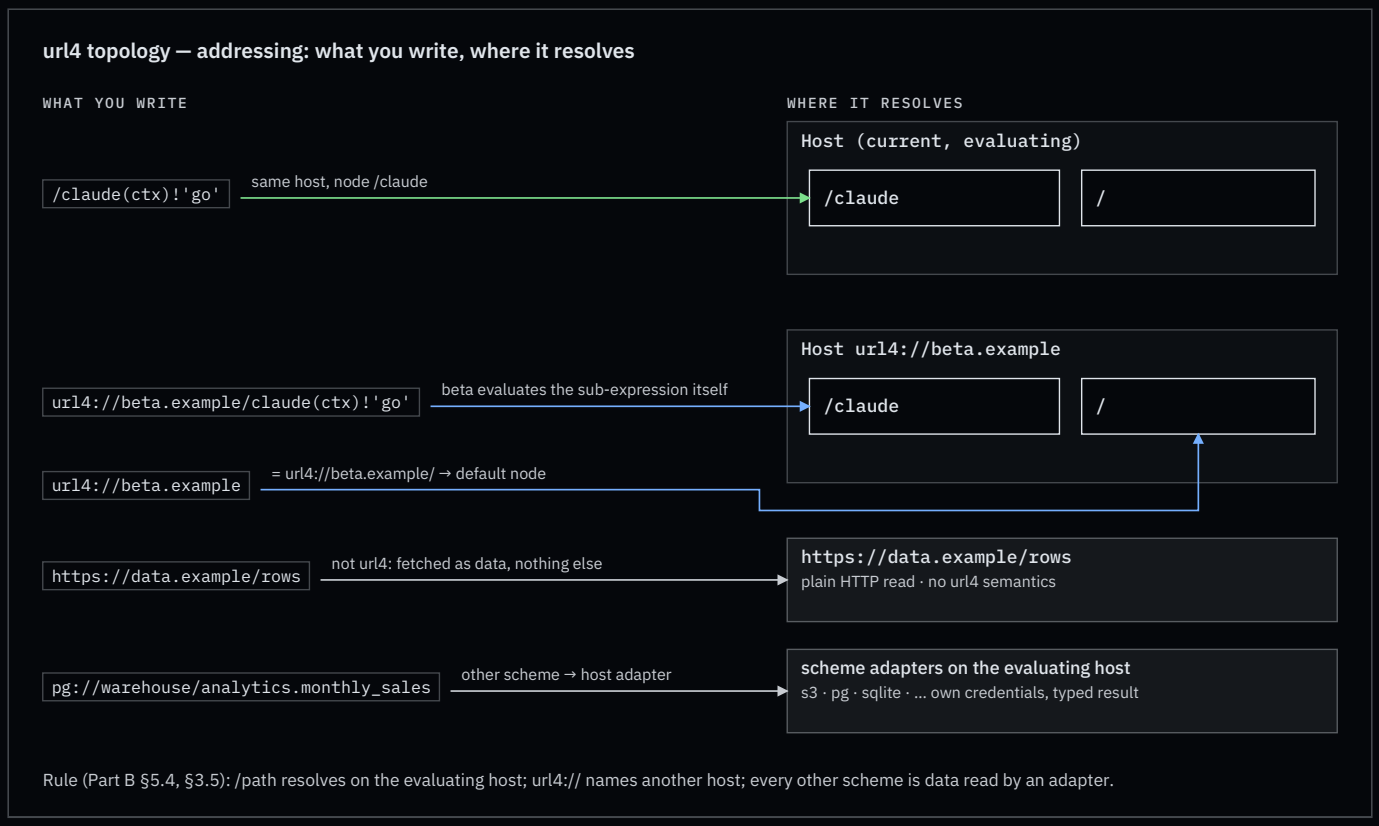
OUR WORD	MEANING	SPEC WORD TODAY	IN THE SDK	IN THE ENGINE
Host	An origin that mounts nodes at paths, serves `/` as the default node, and answers discovery.	Node (Part A §1.4)	`Url4Node` (the class name lags; Appendix B)	the App, after this change
Mount	The binding of a path to a node implementation: `local` (in-process), `command` (subprocess), or `proxy` (forwards to a declared remote node).	not defined	`endpoint()`, `[commands]`, —	in-process handlers only
Evaluator	Whatever runs an expression: resolves sources, fans out, reduces, runs the intent. Every node contains one.	"the node executes" (Part A §1)	`url4.dag.run`	`Url4Executor` in the Runner
Intent processor	The thing inside a node that turns resolved context plus intent into a result: a model call, a script, a command.	Intent processor (Part A §1.4)	endpoint handler	one `httpx` call to aigateway
Requestor	Whoever sends an expression.	Requestor (Part A §1.4)	`Client`	product client
Request tree	The hosts and nodes one expression touches. Strict tree, never a graph (Part H §29.1).	"request tree", "call tree" (Part H §31)	the compiled DAG, per node	one run
Capabilities document	JSON a host publishes to say which nodes, mounts, features and delivery modes it offers.	Part G §27.2 (draft; single node, `collections`, `intent_processors`)	none	none

Words we do not use in the protocol: ensembler, orchestrator, swarm, composition, plan, fusion. "Fusion" stays product copy.

Naming note: Parts C–I on the `url4-refactor` branch still use the older `abc://` scheme and `ABC-*` headers; Parts A/B use `url4://`. This document writes `url4` and `URL4-*` throughout and assumes the rename Kevin lists as his open question 19 (Part I §43).

2. Addressing

Three address forms, plus any other scheme as data. The spec fixes their meaning in Part B §3.1.1, §3.5 and §5.4; we add one rule about the root and one about other schemes.



- `/claude(ctx)!'go'` — a node on the host that is evaluating. Resolves to `url4://<current host>/claude` (Part B §5.4).
- `url4://beta.example/claude(ctx)!'go'` — a node on another host. That host evaluates the sub-expression itself (Part B §3.1.1; SDK invariant OME-535).
- `url4://beta.example` — the host's default node. **Our rule:** `url4://host` ≡ `url4://host/`.
- `https://data.example/rows` — plain HTTP data. No url4 headers, no envelope, `@` is an error (Part B §3.5).

Root and version. `/` is the host's default node. The spec makes `/v1` the protocol-version path (Part D §19.1) but its own examples already write `abc://thepost?q=…` with no path at all (Part I §41.27). We keep both: `/` serves the current version; `/v1` is an explicit alias. Two tensions to settle with Kevin: Part D §19.4 gives the `v` parameter precedence over the path, and §19.3 says a nested expression takes its version from the path it targets, which a bare host address does not carry. The SDK's `url4 serve` (`/v1`) and `Client` (default `/v1`) move to `/` (Appendix B).

Scheme inheritance. A bare relative data URI inherits the scheme of the request that carried it (Part B §5.4.1). Under `url4://` it is a url4 read; under `https://` it is a plain read.

Any scheme is a source. The spec fixes the roles of `url4://` (evaluate) and `https://` (read), lists `s3://` with the rule "the node uses its own credentials", and fails unknown schemes with `unsupported_mode` (Part B §3.5). We generalise the `s3://` rule: any other scheme is a **read through a scheme adapter that the evaluating host mounts**. The SDK already has the slot: `FetchRequest.kind` is `url4`, `http`, `relative`, or `other`.

```
(sales=pg://warehouse/analytics.monthly_sales,
 logo=s3://brand-assets/logo.png;accept=image/png,
 notes=sqlite:///srv/app/notes.db/notes,
 policy=https://data.example/policy.md)!'Draft the Q3 update. Use the logo.'
```

Rules:

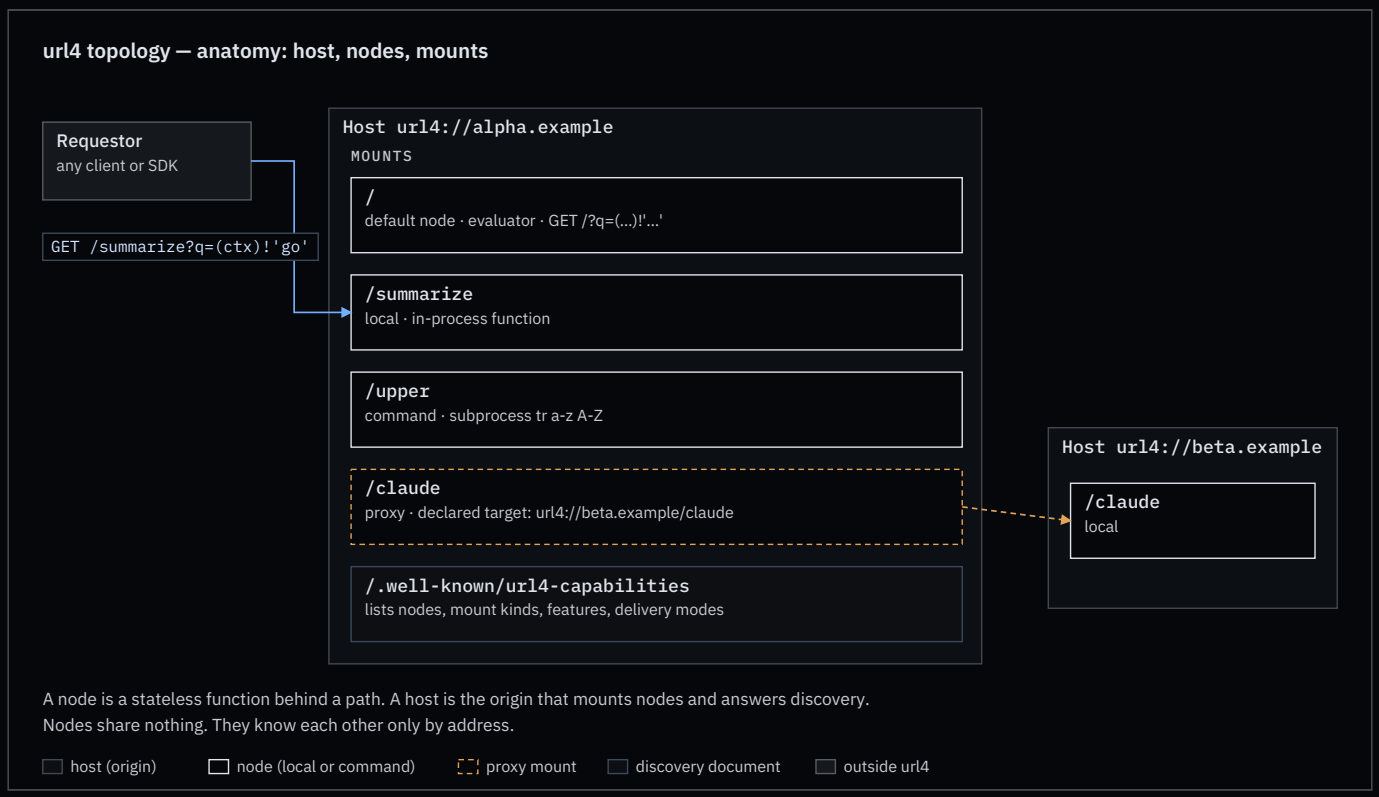
- **The evaluating host resolves it, with its own credentials.** `pg://warehouse` names a connection the host knows; the expression never carries a secret, because the expression is the shareable audit artifact (Part A §1.2). The spec's delegated-credential token can only target `abc_node` or `http_source` (Part H §31.4), so host-owned credentials are the only way a non-HTTP scheme gets any.
- **The adapter types the result** (§8): an S3 object carries its stored `Content-Type`; a table or query returns `application/json` rows, or `text/csv` when asked with `;accept`. Rows are a collection, so `pg://warehouse/analytics.monthly_sales*(row)!'summarise $item'` iterates them (Part B §5.3).
- **Advertised, or refused.** The host lists its schemes in the capabilities document; an unlisted scheme is `unsupported_mode`, permanent (Part B §3.5). Access control and consent apply as for any source (Part B §5.6.4). Such a source is not url4-aware: the envelope marks it `abc_aware: false` and attribution stops at it (Part G §26.4), the same as for any plain HTTP read.
- **Hexagonal.** A scheme adapter is an `IOLayer` adapter registered by scheme; the core never imports it. `url4 serve` gains a `[schemes]` table beside `[data]`.

What a scheme's path means (bucket and key; database, schema and table; file and table) is the adapter's contract, not the grammar's. The grammar only sees `scheme://authority/path`.

---

3. Node contract

A node is a function. That is the whole idea.



A node:

- **MUST** answer `GET <path>?q=<expression>` and evaluate the expression. Every node evaluates url4; a node may walk a DAG before it reaches its own intent processor. There is one grade of node, not two.
- **MUST** be stateless per invocation. It keeps nothing between calls. Its own data is reached through `@` (Part B §5.6) and lives behind it, not in it. The one exception the spec defines is an agent session (Part G §28): the coordinating node holds the session and its transcript for the session's life; participant nodes stay stateless because the transcript travels with every turn. `;coord=` selects a coordination mode (`session`, `debate`, `roundrobin`, `freeform`); the session id is node-issued (`Session-Id` header), not requestor-supplied.

- **MUST** return the envelope (Part D §17) and state the delivery mode it actually used (Part C §11.4).
- **MUST NOT** resolve `@` inside a sub-expression addressed to another host (Part B §5.6.3.1).
- **MUST** answer sync. **SHOULD** offer stream (SSE), WebSocket, and async, and advertise which (§5, §6). When asked for more than it has, it answers with the richest mode it does have; that is never an error.
- **MAY** be mounted on any host, or be its own origin. A Lambda-style function with its own URL is a host with one node.
- Speaks GET. A WebSocket, when offered, is the same GET with `Upgrade: websocket` (§6). Other verbs are 405, except the ones this document adds: `DELETE` on a run handle (§6) and, if adopted, `OPTIONS` (§5).

Nodes share nothing. A node knows another node only by address, and only because the expression named it.

---

4. Host contract

A host is an origin. It routes; it does not evaluate.

- Mounts one or more nodes at paths. `/` is the default node.
- Mount kinds: `local` (a function in the process), `command` (a subprocess; doctrine N4), `proxy` (forwards to a declared target on another host). These are the spec's intent-processor types under our names: `internal` / `function`, `code`, and `abc_delegate` (Part G §27.3). A proxied relative call is exactly the spec's delegation: the caller has already resolved the context, so what crosses the wire is materialised sources plus the intent, never a raw sub-expression. A `command` mount is code execution and inherits the pinning and signing rules of Part H §36.
- Declares proxy mounts with their targets in its capabilities document. Attribution and consumer disclosure (Part H §29.2.1) need the real target, so proxies are never hidden; a source may also cap redistribution depth or name permitted consumers (Part H §29.2.2), and a proxy target is a consumer.
- Serves discovery (§5) and the version alias `/v1` (§2).
- A local SDK process that mounts remote nodes under local names is a host. That is how "run the ensemble on my laptop, but call `/claude` as if it were mine" works.

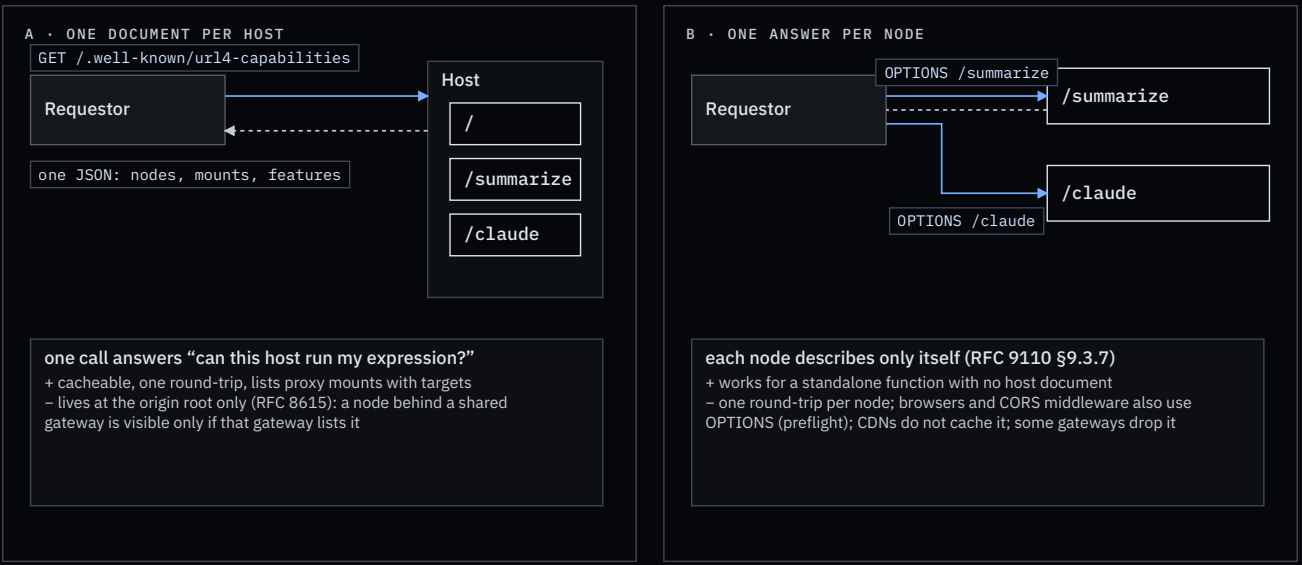
---

5. Discovery — the open decision

Three ways for a requestor to learn what a host can do. The draft Part G specifies the first two: a capabilities document at `/.well-known/abc-capabilities` (SHOULD) and a `Capabilities` response header that MAY point to it (Part G

§27.1). OPTIONS is ours; it appears nowhere in the spec. The spec also reaches for `.well-known` for the policy registry (`/.well-known/abc-policy` , Part H §30.2), so two well-known documents will coexist.

url4 topology — discovery: host document vs per-node OPTIONS



A third mechanism in the draft: a Capabilities response header pointing at the document (Part G §27.1). Section 5 leaves the choice to the owner.
The document is Part G §27.2 (draft, SHOULD); the header is MAY; OPTIONS appears nowhere in the spec.

	A · HOST DOCUMENT `GET /.well-known/url4-capabilities`	B · PER NODE `OPTIONS /<node>`	C · `Capabilities` RESPONSE HEADER
"Can this host run my expression?"	one call, whole answer	one call per node named in the expression	pointer only; still one fetch of A
Standalone function with its own origin	works: the document sits at that origin's root	works	works
Node behind a shared, non-url4 gateway	invisible unless the gateway lists it (RFC 8615: root of the origin only)	works: the node answers for itself	works: any ordinary response can carry the pointer
Caching	ordinary GET: `ETag` , CDN, browser cache	not cached by CDNs or browsers	rides on the cached response
Browsers and CORS	plain GET	preflight uses the same verb; CORS middleware often answers first (RFC 9110 §9.3.7 allows a body, browsers ignore it)	plain header
Gateways and serverless platforms	fine	some strip or auto-answer OPTIONS	fine
Proxy mount targets (attribution)	listed in one place	per node	via A
Spec status	Part G §27.1, SHOULD; schema Part G §27.2	not in the spec	Part G §27.1, MAY

Both share one hazard: the Engine's JWT header is called `URL4-Capability` . The document is "capabilities". Appendix B proposes renaming the header.

Capabilities document: the spec's shape, plus our additions. Part G §27.2 already defines the top level: `abc_version`, `node` (one canonical URI: our host), `self_ref_support`, `collections[]` (`id`, `path`, `description`, `element_schema`, `formats`, …), `behaviors`, `intent_processors[]` (`id`, `type` ∈ `internal` | `abc_delegate` | `http_delegate` | `code` | `function`, `endpoint`, `capabilities`), `processor_policy`, `mode_support`, `agent_profiles`. Content types come from Part F §25.4 as `consumes` / `produces`. Sub-paths are read as collections (Part G §27.4). Our additions, marked, are what makes a processor addressable as a node:

```
{
  "abc_version": 1,
  "node": "url4://alpha.example",
  "default_processor": "summarize",                       // ours
  "intent_processors": [
    { "id": "summarize", "type": "internal", "path": "/summarize",   // path: ours
      "consumes": ["text/plain"], "produces": ["text/plain"],
      "delivery": ["sync", "stream"] },                                 // delivery: ours
    { "id": "upper",     "type": "code",     "path": "/upper" },
    { "id": "claude",    "type": "abc_delegate", "path": "/claude",
      "endpoint": "url4://beta.example/claude" }
  ],
  "schemes": ["s3", "pg", "sqlite"],                        // ours (§2)
  "collections": [ { "id": "science", "path": "/science", "self_ref_eligible": true } ],
  "self_ref_support": true
}
```

The collision to settle with Kevin: the spec's document describes one node with sub-paths as collections; ours needs a path per processor. `intent_processors[].path` is the smallest change that gives both.

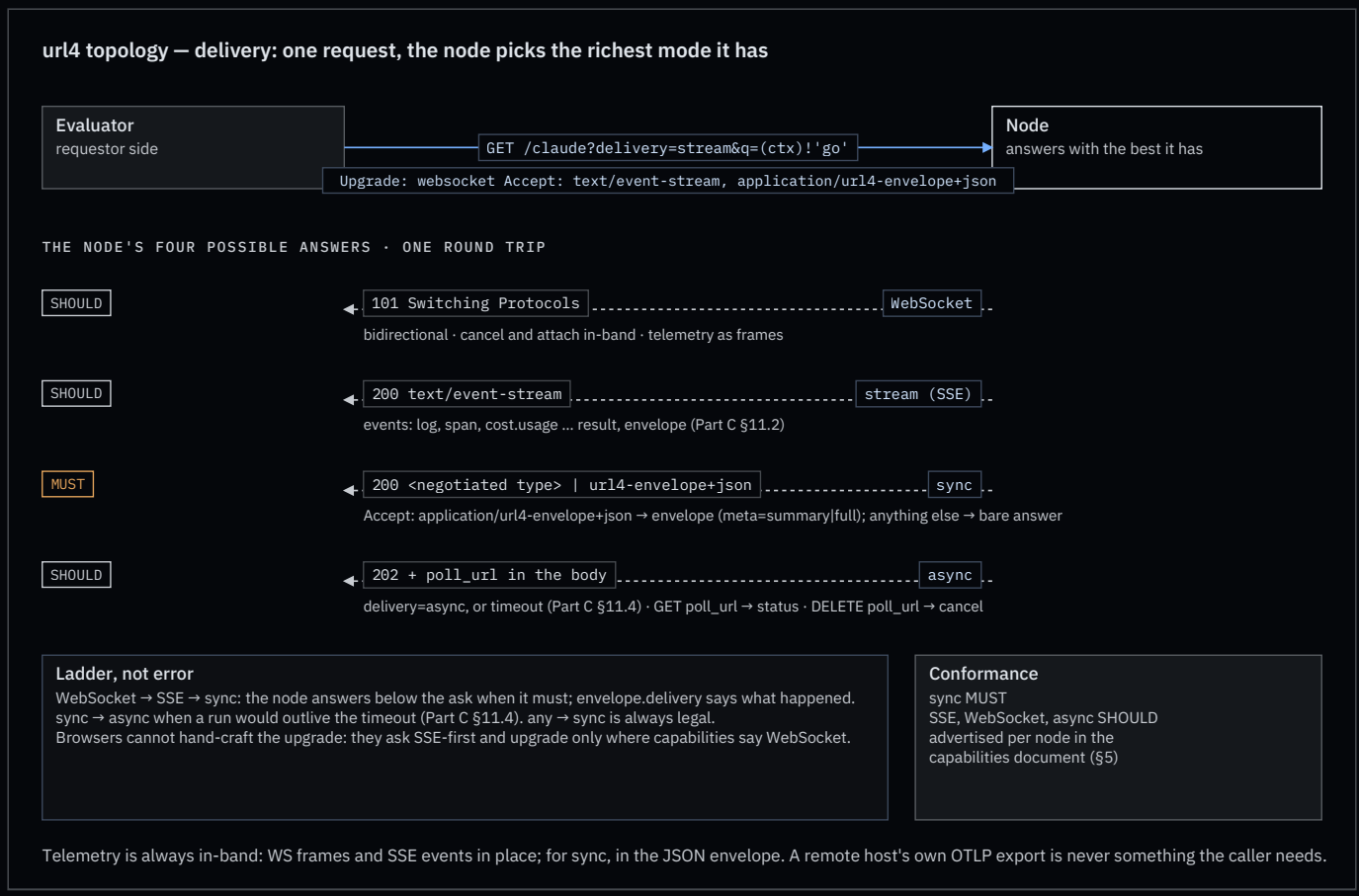
> **Owner decision (open).** Pick one:
>
> ☐ **A only** — host document is a MUST; nothing per node.
>
> ☐ **B only** — OPTIONS per node is a MUST; no host document.
>
> ☐ **A MUST + C SHOULD** — the spec's pair, one level stronger than its SHOULD/MAY.
>
> ☐ **A MUST + B SHOULD** — the document indexes the host; a node may also answer OPTIONS with its own entry.
>
> Recommendation, revised after reading Part G: **A MUST + C SHOULD**, B dropped. The `Capabilities` header solves the gateway case that motivated OPTIONS, and it is already in the draft.

---

6. Delivery and transport

The spec defines three delivery modes and a fallback rule (Part C §11), and a graceful-degradation rule that a node MUST degrade rather than fail and MUST report what it did (Part C §10.2). We adopt them, add WebSocket as a fourth

answer, and make the whole ladder one request. Part I §43 question 26 rules `ws://` sources out of scope; that is a different question from a WebSocket answer on the node's own GET.



One request, the node picks. The evaluator sends the richest ask it can, once:

```
GET /claude?delivery=stream&q=(ctx)!'go'
Upgrade: websocket
Accept: text/event-stream, application/json
```

The node answers with the best it supports. RFC 6455 lets a WebSocket handshake ride an ordinary GET; RFC 9110 lets a server ignore `Upgrade` and answer normally. So one round trip covers the whole ladder:

NODE ANSWERS	MEANING	TELEMETRY TRAVELS AS
`101 Switching Protocols`	**WebSocket**: bidirectional; cancel and attach in-band	frames, same names as the SSE events
`200 text/event-stream`	**stream**: SSE on the same GET (Part C §11.2)	SSE events, then `result`, then `envelope`
`200 text/plain` or `application/json`	**sync**: bare answer, or the envelope, by `Accept`	envelope `meta` (below)
`202` + `poll_url`	**async**: `delivery=async`; the 202 body carries `job_id`, `rid`, `poll_url` (Part C §12.5); `GET` it for status, `DELETE` it to cancel	on the handle

Rules:

- **sync is the only MUST.** SSE, WebSocket and async are SHOULD, advertised per node in the capabilities document (§5). A serverless function that offers only sync and SSE is conformant.

- **Ladder, not error.** WebSocket → SSE → sync. A node answering below the ask is normal; the envelope's `delivery` field says what happened (Part C §11.4), and the degradation is reported like any other (Part C §10.2.2). The spec's ladder has two edges, `stream → sync` and `sync → async`; the WebSocket rung and `any → sync` are ours.
- **Async is the spec's third `delivery` value.** It is requested with `delivery=async` (Part C §9.1), or entered by the node when a sync run would outlive the timeout (Part C §11.4 `sync → async`). The Engine's `Prefer: respond-async` becomes an alias. The run handle is the `poll_url` field of the 202 body; a `Location` header mirroring it is our addition.
- **Browsers go SSE-first.** A browser cannot attach `Accept` to a WebSocket handshake, so a browser evaluator asks for SSE and upgrades only where the capabilities document says WebSocket is offered.
- **The Engine's product session** keeps its WebSocket at `/ws`. It is one instance of the WebSocket rung, not a separate protocol.
- **Telemetry is always in-band.** Whatever the mode, the caller receives logs, spans and cost on the same connection or in the envelope. A remote host's own OTLP export is its business, never something the caller depends on (§7).

Telemetry per mode. WebSocket and SSE carry it in place: an SSE body is a sequence of named events (`event: log`, `event: span`, `event: cost.usage`, then `event: result`, `event: envelope`), in order, on the one response. The spec already builds on this (Part C §12.5 is an SSE event catalog); our three signals are three more names. What SSE lacks against WebSocket is only the return path: no in-band cancel or attach.

Sync has no room for that: the body is the answer. The spec always returns the JSON envelope and uses `Accept` to negotiate the result's content type (Part C §9.1, Part F §25.2). We add one wrapper switch on top, named so it cannot collide with that:

KNOB	GOVERNS	VALUES
`Accept` header	the wrapper, by one dedicated type	`application/url4-envelope+json`: the envelope (Part D §17). Anything else: the bare answer in the negotiated content type, no logs, no telemetry, what `url4 serve` returns today. Our addition; the spec has no bare mode
`meta` param	how much the envelope carries	`none`: result and status. `summary`: counts, latency, cost. `full`: per-source detail, nested child envelopes (Part D §17.3), and a `telemetry` block with logs and spans. The `telemetry` block is ours; per the Hybrid Rule it is present and `null` at `summary` (Part D §17.2.2)
`fmt` param	the shape of the answer content	`text`, `markdown`, `json` (Part C §9.1); orthogonal, a JSON envelope can wrap a markdown answer

An envelope answer with no `meta` gets `summary` (the spec's default is `none`, Part C §9.1), so asking for the envelope always buys the cheap insight; `meta=full` is the explicit debug switch. What a node exposes is its decision (Part D §18.4): a production node may answer `meta=full` with the `telemetry` block redacted, but it MUST keep the structure and use `null` or `"redacted"`, never drop fields silently. A development node hands over everything. Same envelope, same evaluator code, different policy.

Cancel. The spec calls cancellation a gap and offers two shapes, `DELETE <poll_url>` or a `cancel=<rid>` parameter (Part C §16.2). We take the first: over WebSocket, cancel is in-band; otherwise `DELETE` on the `poll_url`, which the Engine already does. New terminal state `cancelled`; SSE event `request.cancelled`; partial result returned when quorum was already met, as §16.2 asks. Note for Kevin: `status: "cancelled"` is not yet in the status enum of Part D §17.4.

7. Telemetry

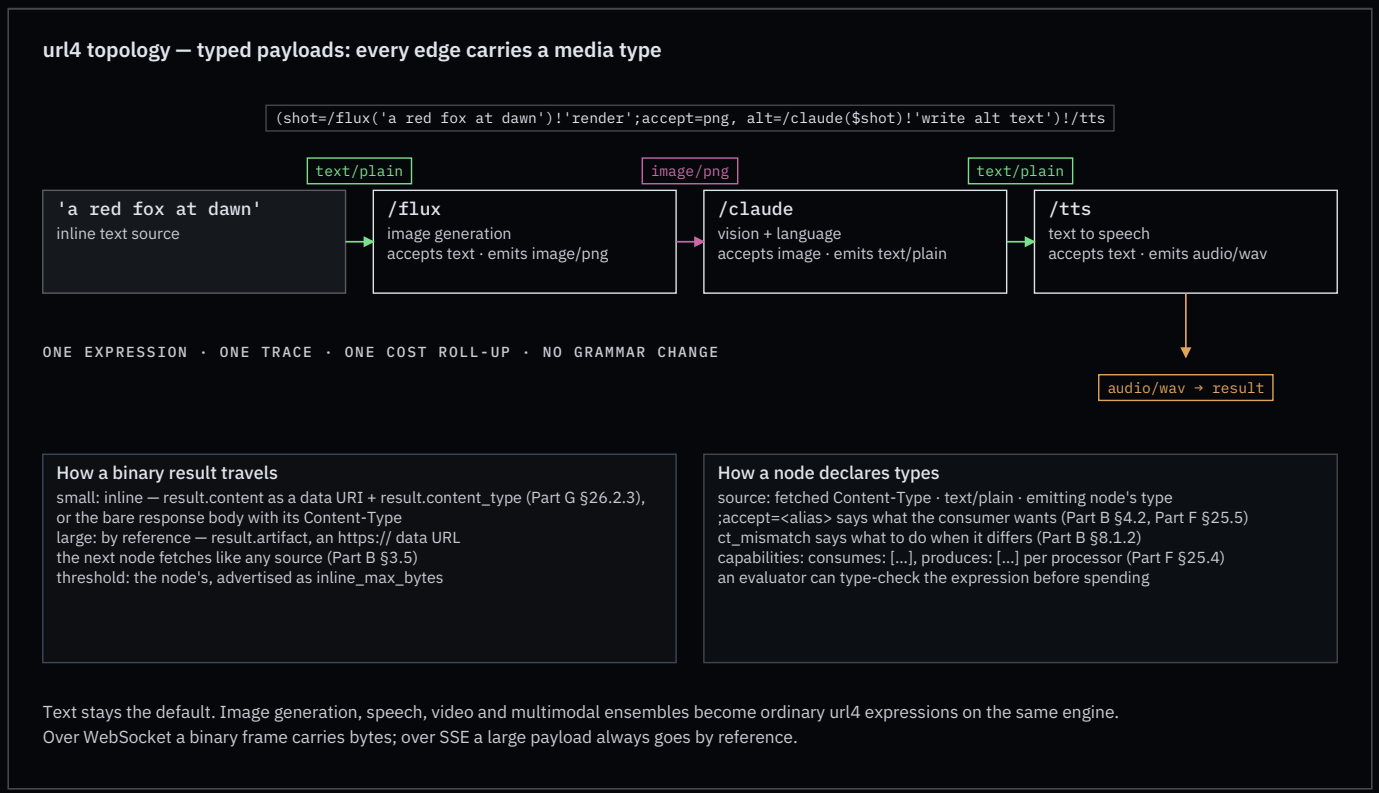
Three signals, one trace, as in the doctrine skill. What changes is where a one-shot GET puts them.

- **sync**: in the body, only when the caller asked for the envelope (`Accept: application/url4-envelope+json` , §6). `meta` sets the level (`summary` by default, `full` adds logs and spans; Part D §17.2). At `meta=full` a child's envelope nests in `source.envelope` (Part D §17.3). Each node aggregates only what it saw itself (Part D §18.1). A bare `text/plain` answer carries nothing.
- **stream** and **WebSocket**: as events: logs, spans, `cost.usage` (self and subtree), then `result` , then `envelope` . Same names in both.
- **durable**: OTLP export to the trace backend. There is no separate "Enclave" store; the exporter superseded it.
- **What the spec already carries**: 21 SSE event types (Part C §12.5), `meta.total_cost` and `budgets_spent` in the envelope (Part D §17.1.2.8, Part E §24), and a node-local audit log (Part H §34). Our logs, spans and `cost.usage` events are additions on top of that catalog, not replacements; cost stays a separate event, and `total_cost` is its roll-up. Every streamed event with content already MUST carry `content_type` (Part C §12.5), which is what §8 builds on.

This closes doctrine item F4. **Idempotency:** a url4 GET is idempotent in the HTTP sense (RFC 9110 §9.2.2): repeating it has no extra server-side effect. Results need not be identical; Part C §15.1 conflates the two (Appendix A). One real exception the spec lists: a repeat that would exceed a consumed budget fails with `budget_exceeded` (Part C §15.2).

8. Typed payloads: a node is a universal inference processor

We have treated a node as text in, text out. That is a habit, not a spec rule. The spec defines a source as "a URI, text, or nested expression" and an intent processor as whatever executes the intent on behalf of the node (Part A §1.4). Nothing says the payload is a string.



Proposal. Every edge in an expression carries a **typed payload**, the way a ComfyUI edge carries an image, a latent or a mask. The type system is the media type: `text/plain` , `image/png` , `audio/wav` , `video/mp4` , `application/json` for embeddings and structured data. Text stays the default and the most common type. No grammar change.

```
(shot=/flux('a red fox at dawn')!'render';accept=png,
 alt=/claude($shot)!'write alt text')!/tts
```

Edges: text → image → text → audio. Three nodes, three processors, one expression, one trace, one cost roll-up.

What the spec already gives us

- `;accept` is a source-level execution annotation (Part B §4.2) that takes a **short alias** (`json`, `csv`, `markdown`, …), never a MIME type, because `/` would read as a relative URI (Part F §25.5). The registry has no image, audio or video rows yet; `png`, `jpeg`, `wav`, `mp4` are proposed in Appendix A. `ct_mismatch` has four values, `fail | ignore | transform_ignore | transform_fail`, default `ignore`, and only applies when `;accept` was declared (Part G §26.3). The spec calls input negotiation out of scope and treats an unusable format as an intent-execution failure, not a source failure (Part F §25.10).
- The spec already has a weighted `image/png` source feeding a text intent (Part I §41.20), and Part G §26.2.3 already says how binary content travels in LLM mode: a data URI, `data:<mediatype>;base64,…`, with raw bytes for RDS intents. The SDK carries a media type on every fetch (`FetchRequest.media_type`) and parses collections by declared type (Part B §5.3.7). The Engine already forks results by size: inline, spilled to a content-addressed artifact, or refused above a hard cap.

Three rules the spec does not yet give (Part F, §25)

1. **A source declares its type.** A URI source has the `Content-Type` it was fetched with; absent that, the node infers from the path or sniffs, and falls back to `application/octet-stream` (Part G §26.2). Inline text is `text/plain`. A nested expression has the `result.content_type` its node emitted (Part D §17.1.2.1). `;accept` says what the consumer wants; `ct_mismatch` says what to do when the two differ.
2. **A binary result travels inline when small, by reference when large.** Bare sync: the response body is the bytes, with its `Content-Type` (Part F §25.5 permits raw binary outside the envelope). Envelope: `result.content` carries a small payload as a data URI with `result.content_type` set, the spec's own form; a large payload becomes `result.artifact`, an `https://` URL that the next node fetches as plain data (Part B §3.5). The inline threshold is the node's, advertised in its capabilities. Over WebSocket a binary frame carries the bytes; over SSE a large payload always goes by reference. Two cautions from the drafts: this is not truncation, which Part G §26.2.1 forbids; and an artifact URL is a redistribution, so it inherits the source's flow constraints and depth cap (Part H §29.2.2).
3. **A processor advertises what it accepts and emits.** The spec's names are `consumes` and `produces` (Part F §25.4), per processor in the capabilities document. An evaluator can then type-check an expression before it spends anything, which is what the plan question in §10 builds on.

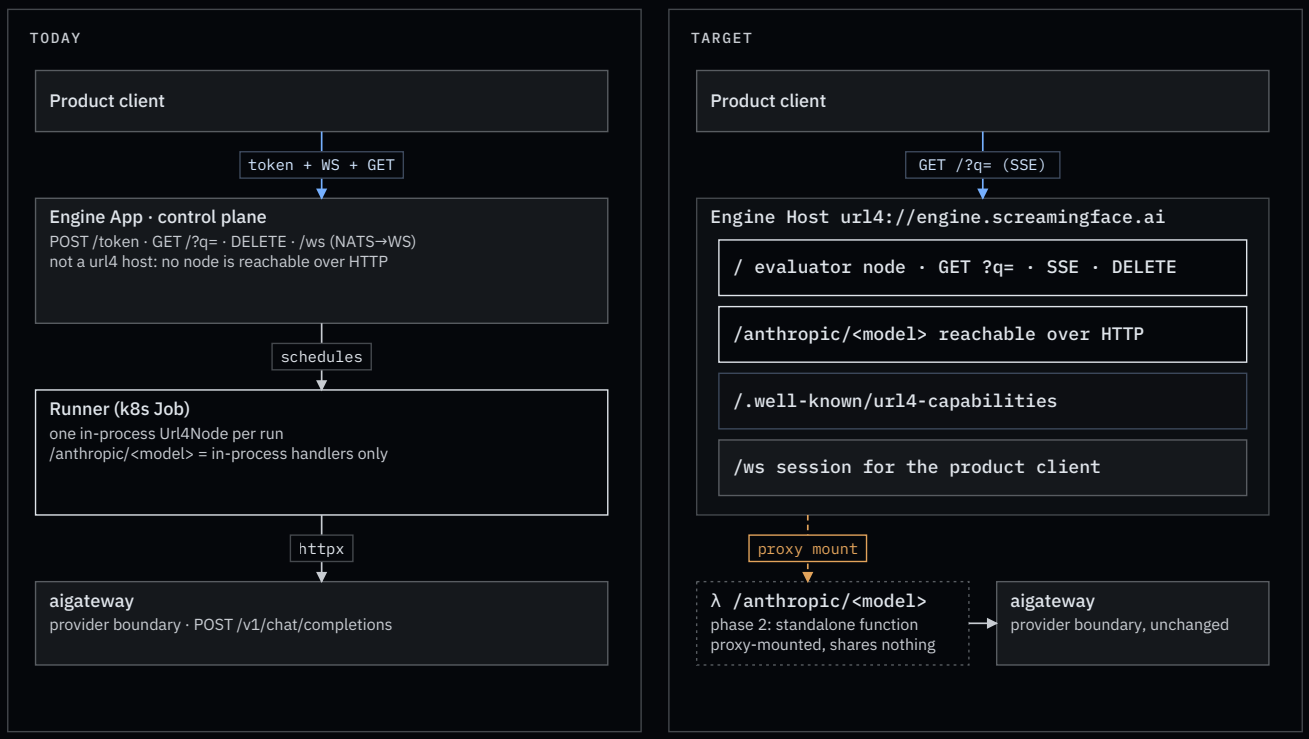
```
{ "id": "flux",   "type": "internal", "path": "/flux",
  "consumes": ["text/plain"], "produces": ["image/png"], "inline_max_bytes": 262144 },
{ "id": "claude", "type": "internal", "path": "/claude",
  "consumes": ["text/plain", "image/png", "image/jpeg"], "produces": ["text/plain"] },
{ "id": "tts",    "type": "internal", "path": "/tts",
  "consumes": ["text/plain"], "produces": ["audio/wav"] }
```

Why it matters. Image generation, text-to-speech, speech-to-text, video evaluation and multimodal ensembles all become ordinary url4 expressions on the same engine, with the same telemetry, the same cost accounting and the same attribution. aigateway stays the provider boundary; it already fronts image and audio providers. The Engine's artifact store is rule 2 as built.

Open. A media type for embeddings (`application/json` with a profile, or a vendor type); how attribution weights and `tokens=` budgets apply to non-text sources (Part E is silent; no `bytes=` or `seconds=` budget key exists). Settled by the draft: `fmt` stays a structure hint and a future version MAY retire it in favour of `Accept` (Part F §25.8).

9. The Engine as a host

url4 topology — the Engine today and as a url4 host



The Runner becomes the evaluator process the / node spawns per run. The App keeps /ws for the product client and gains the node surface.
Serverless posture: nodes are functions; the host is the only thing that needs an origin.

	TODAY	TARGET
App	control plane: `POST /token`, `GET /?q=`, `DELETE`, `/ws` bridged from NATS	a **host**: `/` evaluator node (`GET ?q=` with WebSocket, SSE, sync and async answers, `DELETE`), model nodes over HTTP, capabilities, `/ws` kept for the product client
Runner	k8s Job with one in-process `Url4Node`; model routes are in-process handlers	the evaluator process the `/` node spawns per run
Model nodes	`/anthropic/<model>` reachable only inside the Runner	reachable over HTTP on the host; later each one a standalone function, proxy-mounted
aigateway	provider boundary	unchanged
Node-to-node	unused SDK code	the normal path

Phases. (1) Host surface on the App: discovery per §5, `GET /?q=` answering SSE and sync (WebSocket via `Upgrade` where the App already has it), `DELETE`. (2) Model nodes over HTTP. (3) Standalone model functions, proxy-mounted, listed in the capabilities document.

Serverless posture. A node is a function; it needs no origin of its own. Only the host needs one. The Runner is already function-shaped. The App is not: it holds an audience count in memory, runs two perpetual tasks, and gates runs on an attached WebSocket. Those belong to the product session, not to the node surface, and stay in the App.

10. Deferred, and two questions to move forward

Authentication as a host concern (question, not a decision). Separating Host from Node makes a safe place for authentication possible, because it gives the credentials exactly one owner. The spec's inter-node auth (Part H §31,

"under active development") propagates per-destination encrypted tokens in `ABC-Auth-Token` headers, and Part B §3.5 names a `URL4-Auth-Token` with a target type. The SDK defers all of it: `url4 serve` ships no authn or authz and asks for a reverse proxy in front. The Engine already has the shape we want: the App mints a per-run capability token and the Runner only carries it.

The idea to test: **the host, through its evaluator, is the only party that holds, validates and issues credentials. Nodes never see a raw credential.** A node receives a host-issued session, a capability scoped to one run (`rid` , request tree, purpose, expiry), and uses it for whatever it needs: reading `@` holdings, calling a sibling node, asking the host to fetch an `s3://` source. Outbound, the host forwards the requestor's per-destination tokens and holds the ones addressed to it (Part H §31.5); inbound, the host validates before any node runs. A standalone serverless function is its own host and validates for itself, so the rule holds at every size. The draft already fixes the token's bindings: `rid` , a timestamp with a window of at most 300 seconds, and the destination identity (Part H §31.2).

Questions to answer before this becomes a section:

1. **Session shape.** Is the node-side session the Engine's JWT topic capability generalised (`sub` = run, `iat` window), or the spec's `URL4-Auth-Token` ? One token type, or a host-internal one plus the spec's wire one?
2. **Where the identity lives.** `@alice` access control and consent (Part B §5.6.4) need the requestor's identity at the node. Does the session carry it, or does the node ask the host? Part H §33.1 asks the mirror question, whether identity is per server or per endpoint; the Host/Node split is an answer to it.
3. **Proxy mounts.** For `/claude → url4://beta.example/claude` , which side validates the requestor, and does beta see our identity or a delegated one (Part H §31.2 encrypts to the destination)?
4. **Rename consequence.** Spec §22 addresses tokens to a "node". Under Appendix A delta 1 that becomes a host. Is the token's destination always the host, never the path?
5. **Agent sessions.** The coordinating node issues the `Session-Id` (Part G §28.5). Is that session bound to the run capability, and does the transcript URL it exposes fall under the same access control?
6. **What a node may do with a session.** Only call back into its own host, or reach other hosts directly with a host-minted, destination-bound token?

Recommendation to explore first: host-issued run capability for nodes (the Engine's pattern), spec §22 tokens between hosts, identity carried in the capability claims. Attribution, consent and audit (Parts E and H) then attach to the same run identity.

A plan endpoint on the host (question, reopened). Deferred in the design session; reopened because three later additions give it inputs it did not have: per-node `accepts` and `emits` (§8), per-node `delivery` and `schemes` (§5, §2), and a host egress that already resolves every target's policy before spending (§11). Today the SDK's `Graph.validate()` checks syntax only and the Engine's preflight checks routes on one host.

The idea: a **dry run** on the host. The draft already obliges a node to do most of it: compute the consumer set by static analysis before resolving anything (Part H §29.2.1), be ready to show a source the full call tree (Part H §30.2), and estimate before executing under a hard budget (Part E §24). Steps 3 and 4 of §11 without step 5: parse, resolve mounts, type-check every edge against `accepts` / `emits` , check schemes and delivery modes, consult policy and budgets, and return the request tree with per-source verdicts and an estimated cost. Spend nothing. The answer is the envelope (Part D §17) with `sources[]` and `meta` filled in and no `result` ; a refused source carries its error code, so "can this host run my expression?" becomes one call with a structured no.

Questions:

1. **Shape.** `Prefer: dry-run` on the ordinary GET (the RFC 7240 pattern we already use for `respond-async`), a `plan` protocol parameter beside `q=` , or a separate path? The header keeps the address identical for plan and run, which suits caching and audit.
2. **Does a plan spend?** Policy-registry consults and budget reservations are real calls. Is a plan free by definition, with `meta.total_cost` as an estimate from `pricing_version` , or may it reserve?
3. **Is the plan an artifact?** A plan id returned as a handle, so a later run can say "execute this plan" and skip re-planning. That ties to idempotency (§7) and to the token binding of §11.

4. **Federation.** For a remote subtree, does the host forward `Prefer: dry-run` to the other host and merge its plan, or plan only from that host's capabilities document? The first is exact and costs a round trip; the second is instant and approximate.
5. **Ownership.** The Engine's preflight admission (OME-880) becomes this endpoint's single-host case. Does the SDK gain `url4 plan`, calling the same thing?

Recommendation to explore first: `Prefer: dry-run` on the same GET, envelope-only answer, cacheable like any GET, forwarded to remote hosts that advertise it and approximated from capabilities where they do not.

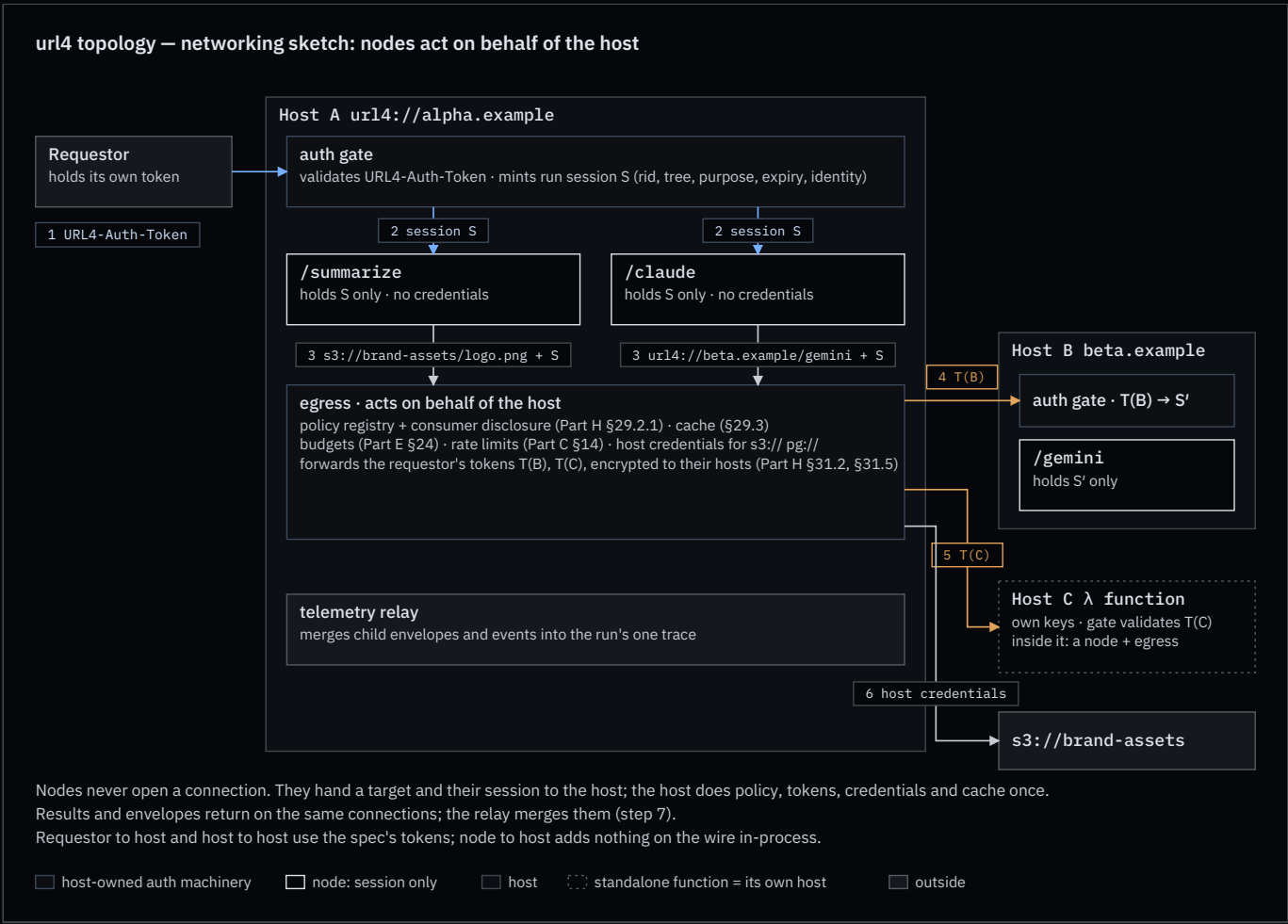
Deferred

- **Swarm** as a protocol noun. Would need membership and trust rules that belong to the unwritten governance parts.
- **Node grades** (processor-only vs evaluator). Rejected: every node evaluates.

---

11. Networking sketch: nodes act on behalf of the host

A brainstorm, not a decision. It answers one question from §10 concretely: **how does a node make an outbound request without holding a credential?**



Three candidate mechanisms

	A · HOST EGRESS	B · DELEGATED TOKEN	C · SIDECAR
Who opens the outbound connection	the host	the node, with a host-minted token	a per-node proxy process

	A · HOST EGRESS	B · DELEGATED TOKEN	C · SIDECAR
Where credentials live	host only; it forwards the requestor's destination-encrypted tokens and holds those addressed to it (Part H §31.5)	the requestor's destination-bound token (Part H §31.2) carried by the node	sidecar only
Policy, disclosure, cache, budgets, rate limits	one place: the host's egress (Part H §29.2.1, §16.3, §20, §31)	at mint time on the host; enforcement split	in the sidecar
Serverless node	nothing to configure	needs network egress and key handling per function	platform-dependent
Cost	an extra hop; large payloads through the host, or by reference (§8)	none	one process per node
What it is today	how the SDK already works: an in-process node fetches through the host's `IOLayer`	the Engine's per-run JWT, generalised	not built

Recommendation to explore first: A for nodes, B between hosts. A node never talks to the network. It talks to its host. The host talks to other hosts with the spec's tokens. A standalone serverless function is its own host, so the same two rules cover it: the parent host reaches it host-to-host (B), and inside it the function is a node using its own host's egress (A). C is a deployment shape of A, not a third protocol.

The flow the diagram draws

1. The requestor calls Host A with its `URL4-Auth-Token` (Part B §3.5). Host A's auth gate validates it.
2. The gate mints a **run session S**: `rid`, the request tree, purpose, expiry, the requestor's identity. Every node in the run receives S and nothing else. In-process nodes get it as an object; command mounts get it in the environment.
3. A node that needs `url4://beta.example/gemini` or `s3://brand-assets/logo.png` hands the target and S to the host's **egress**. It does not open a connection.
4. Egress does the host's work once, in one place: consults the policy registry and discloses consumers (Part H §29.2.1), checks budgets and rate limits (Part E §24, Part C §14), serves from cache when allowed, keyed by requestor identity and consumer set so a shared cache is not a policy bypass (Part H §29.3), and either fetches with the host's own credentials (`s3://`, `pg://`) or forwards the requestor's token **T(B)**, encrypted to Host B, on the sub-request (Part H §31.2, §31.3). The host mints nothing for other hosts: the spec has the originator encrypt every token so that intermediaries cannot read or replace them. What the host mints is internal: the run session S.
5. Host B's gate validates T(B), mints its own session S′, and runs `/gemini`. Its result and envelope come back on the same connection.
6. A proxy-mounted standalone function is Host C: same as step 5 with T(C).
7. Host A's telemetry relay merges what came back into the run's one trace (doctrine F1).

What is new on the wire, and what is not

- Requestor to host, and host to host: nothing new. The spec's tokens and `traceparent`.
- Node to host: **nothing on the wire for in-process nodes**; it is a function call through the host's `IOLayer`, which is what `Url4Node` does today. For out-of-process nodes (command mounts, containers) an explicit host endpoint is needed, something like `GET /.host/egress?u=<absolute URI>` with `URL4-Session: S`. Its path and shape are open.

Open questions this sketch adds

- Does egress return bytes to the node, or a reference (§8)? For large sources, by reference keeps the host out of the data path.

- The spec binds T(B) to `rid`, a timestamp (≤ 300 s) and the destination (Part H §31.2). Does the host-internal session S need the same three bindings?
- Delegated credentials only target `abc_node` or `http_source` (Part H §31.4). For `s3://` and `pg://` the host's own credentials are the only path; should `target_type` widen, or is host-credential-only the rule?
- Can a node ever be granted direct egress (mechanism B at node level)? Probably only for trusted local mounts, and only by host policy.
- Where does the egress endpoint live for command mounts: a Unix socket, a loopback port, or the host's public origin with S as the credential?

12. Alignment with the url4-refactor drafts (2026-09-08)

The `url4-refactor` branch of `OpenMined/screamingface-design` carries draft Parts C–I (cut 2026-04-28 from the v0.4 text, not yet reviewed) and a v0.4 monolith that renumbers everything after §20. This document was first written from the v0.2 monolith. Every citation has been re-anchored to the Part numbering. The table records what the drafts changed, and how this document resolved it.

TOPIC	WE WROTE	THE DRAFT SAYS	RESOLUTION
Capabilities document	schema unwritten	Part G §27.2 specifies it: one `node`, `collections`, `intent_processors`, `processor_policy`, `mode_support`	§5 rewritten as spec shape + marked additions (`path`, `delivery`, `schemes`, `default_processor`)
Discovery mechanisms	`.well-known` vs OPTIONS	`.well-known/abc-capabilities` SHOULD, `Capabilities` header MAY (Part G §27.1); no OPTIONS	header added as mechanism C; recommendation moved to A + C
Sub-paths	`/name` is a node	sub-paths are collections (Part G §27.4); processors are addressed by `id`	proposed `intent_processors[].path` (Appendix A)
Mounts vs delegation	`proxy` forwards a sub-request	`processor=` delegation, five forms, `abc_delegate` receives materialised sources (Part G §27.3)	mounts mapped onto processor types (§4); no new mechanism
Async	`Prefer: respond-async`, `Location` handle	`delivery=async`; 202 body with `poll_url` (Part C §9.1, §12.5)	adopted; `Prefer` and `Location` become aliases
Wrapper switch	`Accept: application/json` = envelope	`Accept` negotiates the result's type; envelope is always JSON (Part C §9.1, Part F §25.2)	dedicated `application/url4-envelope+json` (§6)
Ladder	WS → SSE → sync, ours	two edges only (Part C §11.4); degrade-not-fail is a rule (Part C §10.2)	kept as delta, anchored in §10.2
`;accept` values	MIME types	short aliases only; registry has no image/audio/video rows (Part F §25.5)	examples fixed; alias rows proposed
Binary results	`result.media_type`	`result.content_type`; data URI in LLM mode, raw for RDS (Part G §26.2.3)	adopted; `result.artifact` stays a delta

TOPIC	WE WROTE	THE DRAFT SAYS	RESOLUTION
`ct_mismatch`	fail / convert / pass	four values, default `ignore`, only with `;accept` (Part G §26.3)	adopted
Telemetry events, cost event	ours	21 SSE types (Part C §12.5); `total_cost` scalar, `budgets_spent` (Part D §17, Part E §24)	kept as additions, roll-up mapped to `total_cost`
Statelessness	every node keeps nothing	coordinating node owns an agent session and transcript (Part G §28)	§3 narrowed to participants
Host-minted tokens	egress mints T(B)	originator encrypts every token; intermediaries forward or hold (Part H §31.2, §31.5)	§11 corrected: host forwards, mints only its internal session
Non-HTTP credentials	host's own	`target_type` is `abc_node | http_source` only (Part H §31.4)	stated; question for Kevin
Artifacts and proxies	free to forward	flow constraints and redistribution depth (Part H §29.2.2)	cautions added to §4 and §8
Root `/`	ours	unaddressed; examples use bare `abc://host?q=` (Part I §41.27); `v` param outranks the path (Part D §19.4)	kept; tensions listed in §2
Cancel	`DELETE` on `Location`	`DELETE <poll_url>` or `cancel=<rid>` (Part C §16.2); `cancelled` not in the status enum (Part D §17.4)	`DELETE <poll_url>`; enum gap flagged
Naming	`url4://`, `URL4-*`	C–I still `abc://`, `ABC-*`; consolidation is open question 19 (Part I §43)	kept `url4`; assumption stated in §1

Confirmed by the drafts without change: the three trust relationships and token bindings (Part H §31), the strict request tree (Part H §29.1), `/name` ensembles on one host (Part I §41.9), a weighted image source (Part I §41.20), consumer disclosure before resolution (Part H §29.2.1), the idempotency wording we dispute (Part C §15.1), and every element of our cancel shape (Part C §16.2).

Appendix A — proposed spec deltas for Kevin

Each item names the anchor and the change. All are proposals.

1. **Part A §1.4 — rename.** Node → **Host** ("an origin implementing the protocol; mounts nodes"). Endpoint → **Node** ("a path on a host bound to an evaluator and an intent processor"). Add **Mount** (`local | command | proxy`, proxies declare their target). Consequence to confirm: Part H §31 tokens are then addressed to hosts, never to paths (§10).
2. **Part B §3.1.1, Part D §19 — root.** `url4://host` ≡ `url4://host/`; `/` is the host's default node at the current version; `/v1` is a version alias. Your own examples already do this (Part I §41.27). Also resolve §19.1 (path primary) against §19.4 (`v` outranks path).

3. **Part C §11.2 — bindings.** `delivery=stream` is SSE on the same GET, requested with `Accept: text/event-stream` (§11.2 names SSE but no media type). A node MAY also offer WebSocket by honouring `Upgrade: websocket` on that same GET (`101`); frames carry the same event names. Orthogonal to Part I question 26, which is about `ws://` sources.
4. **Part C §10.2, §11.4 — ladder and floor.** Extend graceful degradation to `delivery` : the node answers with the richest mode it supports, WebSocket → SSE → sync, in one round trip; sync is the only MUST and `any → sync` is always legal. A node answering below the ask is a reported degradation (§10.2.2), not an error. Add a `delivery` list per processor to the capabilities document.
5. **Part C §15.1 — idempotency.** Replace “the protocol does not guarantee idempotency” with: GET is idempotent per RFC 9110 §9.2.2; results are not guaranteed deterministic. §15.2’s `budget_exceeded` row stays as the stated exception.
6. **Part C §16 — cancellation.** Take your first option: `DELETE <poll_url>` ; drop `cancel=<rid>` . Terminal state `cancelled` ; SSE event `request.cancelled` ; propagates to in-flight children; partial result when quorum was met. Add `cancelled` to the status enum in Part D §17.4 and to the state machine in Part C §13.4.
7. **Part D §17 — envelope for sync callers.** A dedicated media type, `application/url4-envelope+json` , selects the envelope; any other `Accept` returns the bare result in its negotiated type. Envelope default `meta=summary` (override of §9.1’s `none`). At `meta=full` the envelope carries a `telemetry` block (`logs[]` , `spans[]` , `cost`), present and `null` at `summary` per the Hybrid Rule (§17.2.2), redactable per §18.4. `total_cost` is the roll-up of a separate `cost.usage` stream event.
8. **Part F §25, Part G §26 — typed payloads.** Add image, audio and video rows with aliases (`png` , `jpeg` , `wav` , `mp4`) to the §25.5 registry; extend §26.2.3’s data-URI rule to results (`result.content` + `result.content_type`); add `result.artifact` for by-reference results above a node-declared `inline_max_bytes` , subject to §29.2.2 flow constraints; WebSocket binary frames MAY carry bytes, SSE goes by reference. Part E: say how weights and budgets apply to non-text sources. Slot §41.24 is free for a multimodal example.
9. **Part B §3.5 — other schemes.** Generalise the `s3://` row: any scheme other than `url4://` , `https://` , `http://` is a read through a scheme adapter on the evaluating host, resolved with the host’s own credentials, typed by the adapter, advertised in the capabilities document (`schemes`), and otherwise `unsupported_mode` (permanent). Path semantics per scheme are the adapter’s contract.
10. **Part G §27.2 — capabilities document.** Keep the schema; add `intent_processors[].path` so a processor is addressable as a node, `delivery` per processor, `schemes` the host reads (§2), `inline_max_bytes` , and a `default_processor` . Rename `abc_*` to `url4_*` with the header consolidation (your question 19).
11. **Part H §31.4 — credential targets.** Either widen `target_type` beyond `abc_node | http_source` , or state that non-HTTP schemes are resolved with the host’s own credentials only (§2).
12. **Part D §17.2.2, §20 — additions must respect the Hybrid Rule and `propagated` .** Our `telemetry` and `session` blocks are present-and-null at `summary` ; unknown fields pass through unaggregated.

Appendix B — follow-up work items

To file after owner review, one per landing.

LANDING	ITEM
screamingface-engine	Host surface phase 1: discovery per §5, `GET /?q=` with SSE, `DELETE` on the run handle
screamingface-engine	Model nodes reachable over HTTP (phase 2); standalone model functions, proxy-mounted (phase 3)
screamingface-engine	Rename the `URL4-Capability` JWT header to avoid the “capabilities” collision

LANDING	ITEM
url4-sdk	Delivery negotiation in `Url4Node.asgi()`: `Upgrade` / `Accept` handling, `application/url4-envelope+json`, SSE body, `delivery=async` with `poll_url`, optional WebSocket answer; evaluator-side single-request ladder in `HttpIOLayer`
url4-sdk	Capabilities document and/or OPTIONS responder, after the §5 decision; advertise `delivery` per node
url4-sdk	`url4 serve` default path `/`; `Client` default path `/`; `/v1` alias
url4-sdk	`proxy` mount kind in `url4.toml`
url4-sdk	Scheme adapters (`s3://`, `pg://`, `sqlite://`, …) as `IOLayer` adapters registered by scheme; `[schemes]` in `url4.toml`; `schemes` in capabilities
url4-sdk	Typed payloads: bytes + media type through `IOLayer` / `FetchRequest`, `;accept` / `ct_mismatch` enforcement, `result.artifact` by-reference fetch
screamingface-engine	Non-text processors over aigateway (image, speech); artifact store as the by-reference path; `accepts` / `emits` in capabilities
url4-sdk	`Url4Node` → host naming retrofit with a deprecation alias
repo	Doctrine skill synced in this unit (T1, N1, F2, F4, term table)
Kevin	Review Appendix A

Appendix C — where the spec lives

- Parts A and B, v0.5 DRAFT (2026-07-10): `secondbrain/kevin-mcdonough/docs/adrs/URL4-Spec-A.md`, `URL4-Spec-B.md`.
- Parts C–I: draft markdown on the `url4-refactor` **branch** of `OpenMined/screamingface-design`, `kevin-mcdonough/docs/adrs/refactor/URL4-Spec-{C…I}.md`, cut 2026-04-28 from the v0.4 text, not yet reviewed, never merged to `main` (where the site shows “Not yet written” stubs). Cited here as “Part X §N”. The same branch holds the v0.4 monolith (commit `f28608a`); the v0.2 monolith this document was first written from is `8a052dc`, and its numbering diverges from §21 on.
- Public docs: `public-docs/src/pages/learn/Url4Page.vue`. They use “fusion” and “typed DAG”; the spec uses neither.
- Doctrine: `.claude/skills/url4-engine/SKILL.md`, updated with this document.